
The Modal Axiom-Schema Architecture

SchemaUnion, ModalSchemaTag, and the unionSound hinge

CSLib — Internal Document

— July 19, 2026 —

Internal document. This report renders the architecture note `docs/modal-axiom-schema-architecture.md` as a typeset deliverable. It is a derived, differently-purposed artifact – the Markdown source remains the primary reference and is unchanged by this document. Definitions, lemma signatures, and axiom schemas are transcribed from the landed, CI-green Lean source cited in each section’s anchors, not from memory.

1 Overview

CSLib’s modal-logic layer formalizes 15 classical normal modal systems — K, T, D, B (a.k.a.

KB), K4, K5, K45, S4, S5, TB, KB5, D4, D5, D45, DB — each as a Hilbert-style proof system over a shared propositional/modal **Proposition** language. Each system’s axiom set was historically a bespoke, hand-written **inductive** with its own 13–18 constructors, byte-identical to its siblings on a shared propositional/K/and-or/diamond-duality core and differing only in which modal-strength schemata (T, D, B, 4, 5) it admits. This document describes the compositional architecture that replaced those hand-written inductives: **15 systems, unified under 14 renamed <Sys>Axiom abbreviations plus S5’s pre-existing ModalAxiom**, all expressed as one-line **abbrevs** over a single combinator, **SchemaUnion**, and a single 18-tag alphabet, **ModalSchemaTag**.

The architecture rests on two libraries, two halves of one abstraction:

- **SchemaUnion.lean** (+ **SchemaTags.lean**) — the *syntactic* side: the tag alphabet, the existential “schema = set of instances” meaning function, the **SchemaUnion** combinator, and the elimination lemmas that make the modal cube’s $\subseteq$ -order a **decide**-able computation on **Finset ModalSchemaTag** values.
- **FrameCorrespondence.lean** (+ **SchemaSoundness.lean**) — the *semantic* side: the five frame-condition-to-validity correspondence lemmas for the modal-strength differentiators, and the master soundness lemma, **unionSound**, hinging the two sides together.

Two payoffs fall out directly: **subsumption** collapses from 24 hand-written per-edge lemmas to one generic **SchemaUnion.subsumption** application discharged by **decide**-able **Finset.subset** facts (Section 2), and **soundness** collapses from 15 per-system case-splits to one **unionSound** application consuming an 18-entry **FrameValidatesTag** table plus the five frame-correspondence lemmas (Section 3, “**unionSound**: The Syntax/Semantics Hinge”).

2 The Schema-Tag Alphabet: ModalSchemaTag, .Holds, and SchemaUnion

Durable anchor: `Cslib/Logics/Modal/ProofSystem/SchemaUnion.lean`.

Every one of the 15 classical systems’ axiom set is built from the same finite vocabulary of axiom *schemata*. **ModalSchemaTag** names that vocabulary as an 18-constructor alphabet, grouping into five roles ($4 + 1 + 5 + 6 + 2 = 18$):

Role	Tag	Schema
Propositional core	implyK	$\phi \rightarrow (\psi \rightarrow \phi)$
	implyS	$(\phi \rightarrow (\psi \rightarrow \chi)) \rightarrow ((\phi \rightarrow \psi) \rightarrow (\phi \rightarrow \chi))$
	efq	$\perp \rightarrow \phi$
	peirce	$((\phi \rightarrow \psi) \rightarrow \phi) \rightarrow \phi$
K-distribution	modalK	$\Box(\phi \rightarrow \psi) \rightarrow (\Box\phi \rightarrow \Box\psi)$
Modal-strength differentiators	modalT	$\Box\phi \rightarrow \phi$
	modalD	$\Box\phi \rightarrow \Diamond\phi$
	modalB	$\phi \rightarrow \Box\Diamond\phi$
	modalFour	$\Box\phi \rightarrow \Box\Box\phi$
	modalFive	$\Diamond\phi \rightarrow \Box\Diamond\phi$
And/or	andI	$\phi \rightarrow (\psi \rightarrow (\phi \wedge \psi))$
	andE1	$(\phi \wedge \psi) \rightarrow \phi$
	andE2	$(\phi \wedge \psi) \rightarrow \psi$
	orI1	$\phi \rightarrow (\phi \vee \psi)$
	orI2	$\psi \rightarrow (\phi \vee \psi)$
	orE	$(\phi \rightarrow \chi) \rightarrow ((\psi \rightarrow \chi) \rightarrow ((\phi \vee \psi) \rightarrow \chi))$
Diamond duality	diaDualityFwd	$\Diamond\phi \rightarrow \neg\Box\neg\phi$
	diaDualityBack	$\neg\Box\neg\phi \rightarrow \Diamond\phi$

Table 1: The 18 `ModalSchemaTag` constructors, grouped by role. Schemas are transcribed verbatim from each constructor’s doc comment in `SchemaUnion.lean`.

A tag by itself is just a label; `ModalSchemaTag.Holds` gives each tag its formula-level meaning as an existential over the schema’s metavariables — the “schema = set of its instances” encoding used throughout:

```
def ModalSchemaTag.Holds : ModalSchemaTag → Proposition
  Atom → Prop
  | .implyK, χ => ∃ φ ψ : Proposition Atom, χ = φ.imp
    (ψ.imp φ)
  | .modalT, χ => ∃ φ : Proposition Atom, χ =
    (Proposition.box φ).imp φ
  | .modalB, χ => ∃ φ : Proposition Atom, χ = Axioms.AxiomB
    φ
  -- ... 15 further clauses, one per remaining tag
```

Every clause's proposition shape matches, byte-for-byte, the corresponding constructor of the pre-existing per-system axiom inductives, which is what makes it possible to replace those inductives with a single combinator, parametrized by a chosen `Finset` of tags:

```
def SchemaUnion (S : Finset ModalSchemaTag) : Proposition
  Atom → Prop :=
  fun χ => ∃ t ∈ S, t.Holds χ
```

`SchemaUnion S χ` holds iff `χ` is an instance of some tag in `S`. Because the alphabet is finite and carries `DecidableEq`, choosing `S` to be a concrete `Finset` literal turns each system's axiom predicate into a one-line declaration. Fourteen of the fifteen systems now read `abbrev <Sys>Axiom := SchemaUnion sysTags in ProofSystem/Instances/{K,T,D,B,K4,K5,K45,S4,TB,KB5,D4,D5,D45,DB}.lean`. `S5` is not a structural exception: `Metalogic/DerivationTree.lean` line 69 reads exactly

```
abbrev ModalAxiom : Proposition Atom → Prop := SchemaUnion
  s5Tags
```

so `ModalAxiom` is the 15th system folded into the same one-line-abbrev-over-`SchemaUnion` pattern as the other 14, its name and public API preserved by redefinition-in-place.

Alongside the combinator, three `@[simp]` elimination lemmas give the ergonomic half of the design: a concrete `SchemaUnion sysTags φ` obligation rewrites via `simp` into the named disjunction of its tags' `.Holds` clauses, rather than requiring raw `fin_cases` destructuring at every call site:

```
theorem SchemaUnion.empty_iff {φ : Proposition Atom} :
  SchemaUnion (∅ : Finset ModalSchemaTag) φ ↔ False

theorem SchemaUnion.insert_iff {t : ModalSchemaTag} {S :
  Finset ModalSchemaTag}
  {φ : Proposition Atom} :
  SchemaUnion (insert t S) φ ↔ t.Holds φ ∨ SchemaUnion S
  φ

theorem SchemaUnion.union_iff {Sa Sb : Finset
  ModalSchemaTag} {φ : Proposition Atom} :
  SchemaUnion (Sa ∪ Sb) φ ↔ SchemaUnion Sa φ ∨
  SchemaUnion Sb φ
```

Durable anchors: `SchemaUnion.lean` (`ModalSchemaTag`, `.Holds`, `SchemaUnion`, the three `@[simp]` lemmas); `Metalogic/DerivationTree.lean` (`ModalAxiom`, line 69); `ProofSystem/Instances/{K,...,S5}.lean` (the 15 per-system instance-registration files).

3 Subsumption as `Finset.subset`: The Syntactic Modal Cube

Durable anchors: `SchemaUnion.subsumption` (`SchemaUnion.lean`), `SchemaTags.lean` (the 15 per-system tag sets), `MetaLogic/InterSystem/AxiomSubsumption.lean` (the 24 direct-edge lemmas).

The modal cube — the familiar inclusion order $K \subseteq T \subseteq S4 \subseteq S5$, and its 11 further systems and their edges — used to be witnessed by 24 hand-written, per-edge subsumption lemmas, each proved by a bespoke case analysis over the source system’s axiom inductive. `SchemaUnion` replaces all 24 with one generic lemma:

```
theorem SchemaUnion.subsumption {Sa Sb : Finset
  ModalSchemaTag} (hsub : Sa  $\subseteq$  Sb)
  { $\phi$  : Proposition Atom} (h : SchemaUnion Sa  $\phi$ ) :
  SchemaUnion Sb  $\phi$ 
```

Enlarging the tag set only weakens the resulting predicate — the proof is three lines (`obtain (t, ht, h $\phi$ ) := h; exact (t, hsub ht, h $\phi$ )`). Every `XAxiom_implies_YAxiom` lemma across the cube is now `SchemaUnion.subsumption`

(by `decide`) `h`: a decide-able `Finset.subset` fact, discharged automatically because `ModalSchemaTag` is a finite `DecidableEq` type and every system’s tag set is a concrete `Finset` literal built from `insert/kCore`. This is the framing point of the whole design: **the modal cube’s $\subseteq$ -order IS the $\subseteq$ -order on tag sets** — subsumption is no longer a proved fact about 15 independent inductives, it is a computation on 15 `Finset` values.

3.1 The 15 tag sets

Each of the 15 per-system tag sets (`SchemaTags.lean`) is the shared 13-tag core, `kCore`, unioned with a subset of the 5 modal-strength differentiator tags:

```
def kCore : Finset ModalSchemaTag := -- 13 tags: implyK,
  implyS, efq, peirce,
  ..., -- modalK, andI,
  andE1, andE2, orI1, -- orI2, orE,
  diaDualityFwd, diaDualityBack
```

System	Tag set
K	kCore
T	kCore $\cup$ {modalT}

System	Tag set
D	$\text{kCore} \cup \{\text{modalD}\}$
B (KB)	$\text{kCore} \cup \{\text{modalB}\}$
K4	$\text{kCore} \cup \{\text{modalFour}\}$
K5	$\text{kCore} \cup \{\text{modalFive}\}$
K45	$\text{kCore} \cup \{\text{modalFour}, \text{modalFive}\}$
S4	$\text{kCore} \cup \{\text{modalT}, \text{modalFour}\}$
S5	$\text{kCore} \cup \{\text{modalT}, \text{modalFour}, \text{modalB}\}$
TB	$\text{kCore} \cup \{\text{modalT}, \text{modalB}\}$
KB5	$\text{kCore} \cup \{\text{modalB}, \text{modalFive}\}$
D4	$\text{kCore} \cup \{\text{modalD}, \text{modalFour}\}$
D5	$\text{kCore} \cup \{\text{modalD}, \text{modalFive}\}$
D45	$\text{kCore} \cup \{\text{modalD}, \text{modalFour}, \text{modalFive}\}$
DB	$\text{kCore} \cup \{\text{modalD}, \text{modalB}\}$

Table 2: The 15 per-system tag sets, each `kCore` unioned with a subset of the 5 differentiator tags (`SchemaTags.lean`).

3.2 The syntactic modal cube

The 24 direct edges in `AxiomSubsumption.lean` — each now a one-line `SchemaUnion.subsumption (by decide) h proof`, where previously each was a hand-written per-edge `match/cases` lemma — form a layered DAG on the 15 tag sets, ordered by number of differentiators:

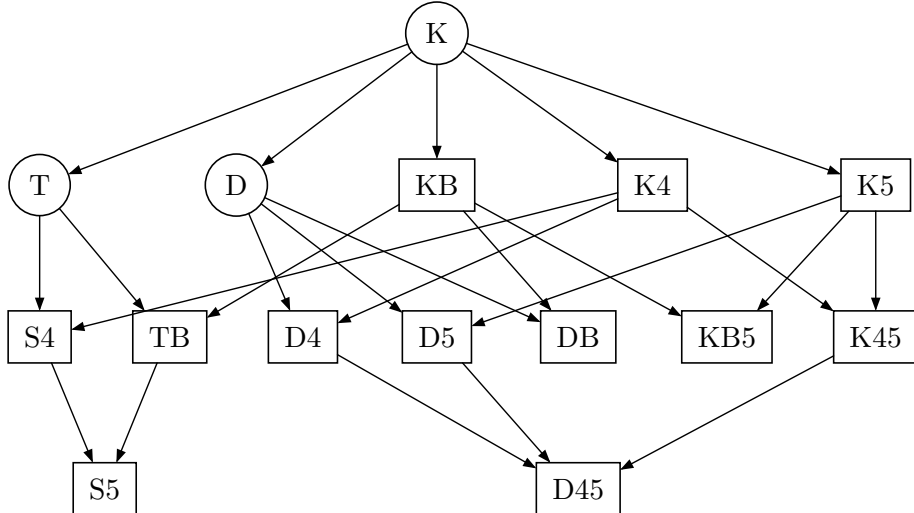


Figure 1: The 24-edge syntactic modal cube (`AxiomSubsumption.lean`), layered by differentiator count. Every edge is a `decide`-able

`Finset.subset` fact between the two systems’ tag sets (Table 2). Notably absent: $\text{KB5} \rightarrow \text{S5}$ (Section 5, “ $\text{S5} = \text{T} + 4 + \text{B}$ ”).

Remark

Reading the diagram

Row 0 is `kCore` alone (K); row 1 adds one differentiator; row 2 adds two; row 3 (S5, D45) adds three. Every edge points from a smaller tag set to a `Finset.insert/u`-larger one. The two into-S5 edges (from S4 and TB) are the only edges terminating at row 3 from a single-parent T/4/B combination; D45 gathers edges from all three of its two-differentiator predecessors (K45, D4, D5).

3.3 Disambiguation: this is not `Cslib/Logics/Modal/Cube.lean`

CSLib has a second, pre-existing artifact also called “the modal cube”: `Cube.lean` defines each classical system as a **semantic** frame-class — a `Set (Model World Atom)` characterized by a frame property, e.g.

```
def T World Atom := logic {m : Model World Atom | Std.Refl m.r}
```

`Cube.lean` was authored for a different purpose and predates the schema-union combinator. It must not be conflated with the syntactic tag-set cube of this section:

Artifact	File	Kind	Each system is a...
Semantic frame-class cube	<code>Cube.lean</code>	pre-existing, frame-based	<code>Set (Model World Atom)</code>
Syntactic tag-set cube	<code>AxiomSubsumption.lean</code> + <code>SchemaTags.lean</code>	new, proof-theoretic	<code>Finset</code> <code>ModalSchemaTag</code>

Table 3: The two independent “modal cube” artifacts living side by side. This document is about the syntactic tag-set cube only.